Builder Pattern

The **Builder Pattern** is a creational design pattern that separates the construction of a complex object from its representation. This allows you to create different types and representations of an object using the same construction process.

Formal Definition:
"Builder pattern builds a complex object step by step. It separates the construction of a complex object from its representation, so that the same construction process can create different representations."

In simpler terms:
Imagine you're ordering a custom burger. You choose the bun, patty, toppings, sauces, and whether you want it grilled or toasted. The chef follows your instructions step by step to build your custom burger. This is what the Builder Pattern does — it lets you construct complex objects by specifying their parts one at a time, giving you flexibility and control over the object creation process.

Real-life Analogy (Custom Pizza Order)

Think of ordering a pizza online. You select the crust type, size, toppings, cheese, and sauce — all step by step. The pizza shop then builds your pizza according to your selections. Different customers can use the same process to get entirely different pizzas. This is the essence of the Builder Pattern: a structured, step-wise approach to creating customized complex objects.

Understanding the Problem

Imagine you're building a **BurgerMeal** in your application. A burger must have some **mandatory components** like: Bun and Patty. And it can also include **option components** like: Sides, Toppings, and Cheese.

Now let’s try to implement this using a traditional constructor approach:

```
#include <bits/stdc++.h>

using namespace std;


// Represents a customizable Burger Meal

class BurgerMeal {

   // Mandatory components

   string bun;

   string patty;


   // Optional components

   string sides;

   vector<string> toppings;

   bool cheese;


public:

   // Constructor trying to handle all combinations

   BurgerMeal(string bun, string patty, string sides, vector<string>
toppings, bool cheese) {
```

```
        this->bun = bun;
        this->patty = patty;
        this->sides = sides;
        this->toppings = toppings;
        this->cheese = cheese;
    }
};

int main() {
    // Constructing the object with only required details
    BurgerMeal burgerMeal("wheat", "veg", "", {}, false);
    return 0;
}

import java.util.*;

// Represents a customizable Burger Meal
class BurgerMeal {
    // Mandatory components
    private String bun;
    private String patty;
```

```
    // Optional components
    private String sides;
    private List<String> toppings;
    private boolean cheese;

    // Constructor trying to handle all combinations
    public BurgerMeal(String bun, String patty, String sides, List<String>
toppings, boolean cheese) {
        this.bun = bun;
        this.patty = patty;
        this.sides = sides;
        this.toppings = toppings;
        this.cheese = cheese;
    }
}

class Main {
    public static void main(String[] args) {
        // Constructing the object with only required details
        BurgerMeal burgerMeal = new BurgerMeal("wheat", "veg", null, null,
false);
```

```
    }
}


# Represents a customizable Burger Meal
class BurgerMeal:
    # Constructor trying to handle all combinations
    def __init__(self, bun, patty, sides=None, toppings=None, cheese=False):
        # Mandatory components
        self.bun = bun
        self.patty = patty

        # Optional components
        self.sides = sides
        self.toppings = toppings
        self.cheese = cheese

# Constructing the object with only required details
burger_meal = BurgerMeal("wheat", "veg", None, None, False)
```

Issues in Code

This constructor approach works, but it creates multiple problems:

- **Hard to Read and Maintain:**
  The user has to remember the order of parameters and their types. It becomes difficult to read when more optional parameters are added.
- **Unnecessary null values:**
  Even if the user doesn't want toppings or sides, they still have to pass null explicitly. This clutters the object creation code.
- **Risk of `NullPointerException`:**
  If we forget to null-check before accessing optional values inside the class, it may lead to runtime exceptions.
- **Too Many Constructor Overloads:**
  To handle various combinations (e.g., with cheese, without sides, only toppings, etc.), you'd need to create multiple overloaded constructors — which is not scalable.
- **Tight Coupling Between Parameters and Construction:**
  There is no flexibility to set values step by step. The entire object must be built in one go, which doesn't match the natural way of ordering or customizing a burger.

Telescoping Constructor Anti-Pattern

To manage optional parameters, many developers try to solve this by writing multiple overloaded constructors — each with one more optional parameter than the last. For example:

```
class BurgerMeal {

   public BurgerMeal(String bun, String patty) { ... }

   public BurgerMeal(String bun, String patty, boolean cheese) { ... }

   public BurgerMeal(String bun, String patty, boolean cheese, String side)
{ ... }
```

```
    public BurgerMeal(String bun, String patty, boolean cheese, String side, String drink) { ... }

}
```

This is called **Telescoping Constructor Anti-Pattern**.

But this creates a cascade of constructors that become:

- **Hard to read and write**
- **Error-prone** due to confusing parameter order
- **Difficult to maintain** when more fields are added
- **Inflexible**, as users must use parameters in a specific order

It occurs most commonly in **Java**, which lacks support for **optional or default parameters** (unlike C++ or Python). Because of this limitation, developers are forced to create multiple constructor overloads to handle different combinations of parameters.

Clearly, this approach doesn't scale well. And this is exactly the kind of problem that the **Builder Pattern** is designed to solve. It gives the user full control over which parts to build while keeping the construction code clean, readable, and safe.

The Solution

To solve the problems we saw earlier with constructors, we use the **Builder Pattern**. It separates object construction from its representation, allowing

us to build step-by-step while keeping the object immutable and readable.

Code

```
#include <bits/stdc++.h>

using namespace std;


// Represents a customizable Burger Meal

class BurgerMeal {

private:

    // Required components

    string bunType;

    string patty;


    // Optional components

    bool hasCheese;

    vector<string> toppings;

    string side;

    string drink;


public:

    // Static nested Builder class

    class BurgerBuilder {
```

```
private:
    // Required
    string bunType;
    string patty;

    // Optional
    bool hasCheese = false;
    vector<string> toppings;
    string side = "";
    string drink = "";

    // Give BurgerMeal access to private members
    friend class BurgerMeal;

public:
    // Builder constructor with required fields
    BurgerBuilder(string bunType, string patty)
        : bunType(bunType), patty(patty) {}

    // Method to set cheese
    BurgerBuilder& withCheese(bool hasCheese) {
```

```
    this->hasCheese = hasCheese;
    return *this;
}

// Method to set toppings
BurgerBuilder& withToppings(vector<string> toppings) {
    this->toppings = toppings;
    return *this;
}

// Method to set side
BurgerBuilder& withSide(string side) {
    this->side = side;
    return *this;
}

// Method to set drink
BurgerBuilder& withDrink(string drink) {
    this->drink = drink;
    return *this;
}
```

```
    // Final build method
    BurgerMeal build() {
        return BurgerMeal(*this);
    }
  };

  // Private constructor to force use of Builder
  BurgerMeal(const BurgerBuilder& builder) {
    bunType = builder.bunType;
    patty = builder.patty;
    hasCheese = builder.hasCheese;
    toppings = builder.toppings;
    side = builder.side;
    drink = builder.drink;
  }
};

int main() {
  // Creating burger with only required fields
  BurgerMeal plainBurger = BurgerMeal::BurgerBuilder("wheat",
"veg").build();
```

```
    // Burger with cheese only
    BurgerMeal burgerWithCheese = BurgerMeal::BurgerBuilder("wheat",
"veg")
                                .withCheese(true)
                                .build();

    // Fully loaded burger
    vector<string> toppings = {"lettuce", "onion", "jalapeno"};
    BurgerMeal loadedBurger = BurgerMeal::BurgerBuilder("multigrain",
"chicken")
                                .withCheese(true)
                                .withToppings(toppings)
                                .withSide("fries")
                                .withDrink("coke")
                                .build();

    return 0;
}

import java.util.*;
```

```
// Represents a customizable Burger Meal
class BurgerMeal {
    // Required components
    private final String bunType;
    private final String patty;

    // Optional components
    private final boolean hasCheese;
    private final List<String> toppings;
    private final String side;
    private final String drink;

    // Private constructor to force use of Builder
    private BurgerMeal(BurgerBuilder builder) {
        this.bunType = builder.bunType;
        this.patty = builder.patty;
        this.hasCheese = builder.hasCheese;
        this.toppings = builder.toppings;
        this.side = builder.side;
        this.drink = builder.drink;
    }
```

```
// Static nested Builder class
public static class BurgerBuilder {
    // Required
    private final String bunType;
    private final String patty;

    // Optional
    private boolean hasCheese;
    private List<String> toppings;
    private String side;
    private String drink;

    // Builder constructor with required fields
    public BurgerBuilder(String bunType, String patty) {
        this.bunType = bunType;
        this.patty = patty;
    }

    // Method to set cheese
    public BurgerBuilder withCheese(boolean hasCheese) {
```

```
    this.hasCheese = hasCheese;
    return this;
}

// Method to set toppings
public BurgerBuilder withToppings(List<String> toppings) {
    this.toppings = toppings;
    return this;
}

// Method to set side
public BurgerBuilder withSide(String side) {
    this.side = side;
    return this;
}

// Method to set drink
public BurgerBuilder withDrink(String drink) {
    this.drink = drink;
    return this;
}
```

```
        // Final build method
        public BurgerMeal build() {
            return new BurgerMeal(this);
        }
    }
}

class Main {
    public static void main(String[] args) {
        // Creating burger with only required fields
        BurgerMeal plainBurger = new BurgerMeal.BurgerBuilder("wheat",
"veg")
                                .build();

        // Burger with cheese only
        BurgerMeal burgerWithCheese = new
BurgerMeal.BurgerBuilder("wheat", "veg")
                                .withCheese(true)
                                .build();

        // Fully loaded burger
```

```
        List<String> toppings = Arrays.asList("lettuce", "onion", "jalapeno");
        BurgerMeal loadedBurger = new
BurgerMeal.BurgerBuilder("multigrain", "chicken")
                                .withCheese(true)
                                .withToppings(toppings)
                                .withSide("fries")
                                .withDrink("coke")
                                .build();
    }
}


# Represents a customizable Burger Meal
class BurgerMeal:
    # Private constructor to force use of Builder
    def __init__(self, builder):
        # Required components
        self.bunType = builder.bunType
        self.patty = builder.patty

        # Optional components
        self.hasCheese = builder.hasCheese
        self.toppings = builder.toppings
```

```
        self.side = builder.side
        self.drink = builder.drink

    # Static nested Builder class
    class BurgerBuilder:
        # Builder constructor with required fields
        def __init__(self, bunType, patty):
            # Required
            self.bunType = bunType
            self.patty = patty

            # Optional
            self.hasCheese = False
            self.toppings = []
            self.side = None
            self.drink = None

        # Method to set cheese
        def withCheese(self, hasCheese):
            self.hasCheese = hasCheese
            return self
```

```
    # Method to set toppings
    def withToppings(self, toppings):
        self.toppings = toppings
        return self

    # Method to set side
    def withSide(self, side):
        self.side = side
        return self

    # Method to set drink
    def withDrink(self, drink):
        self.drink = drink
        return self

    # Final build method
    def build(self):
        return BurgerMeal(self)

# Creating burger with only required fields
```

```
plain_burger = BurgerMeal.BurgerBuilder("wheat", "veg").build()


# Burger with cheese only
burger_with_cheese = (
    BurgerMeal.BurgerBuilder("wheat", "veg")
    .withCheese(True)
    .build()
)


# Fully loaded burger
toppings = ["lettuce", "onion", "jalapeno"]
loaded_burger = (
    BurgerMeal.BurgerBuilder("multigrain", "chicken")
    .withCheese(True)
    .withToppings(toppings)
    .withSide("fries")
    .withDrink("coke")
    .build()
)
```

Understanding the Code

- **Private Constructor** The constructor of `BurgerMeal` is made private so that object creation is restricted to the `Builder` only.
- **Nested Static `BurgerBuilder` Class**
  This builder class holds the same fields as `BurgerMeal`. It ensures immutability and keeps construction controlled.
- **Fluent API Style**
  Each method (like `withCheese`, `withSide`) returns the builder itself, enabling method chaining in a fluent and readable manner.
- **Selective Configuration**
  Only required fields (`bunType`, `patty`) are passed to the builder's constructor. Everything else is optional and set via `withXYZ()` methods.
- **Final Step: build()**
  Once all desired fields are set, calling `.build()` finalizes the object construction and returns the `BurgerMeal` instance.

Why This is Better

Aspect	Constructor Approach	Builder Pattern
Object readability	Poor (nulls, long argument list)	Excellent (fluent and expressive)
Flexibility	Low (all-or-nothing setup)	High (configure only what's needed)
Maintainability	Hard to scale with more fields	Easy to extend with more options

Safety	High chance of errors with nulls	Controlled and safe instantiation

---

When to Use and When to Avoid the Builder Pattern

When to Use?

You should consider using the Builder Pattern in the following scenarios:

- **An object has multiple fields,** especially when many of them are optional. Managing such objects using constructors becomes messy and error-prone.
- **Immutability is preferred -** Builder lets you construct an object step by step and then make it immutable once built.
- **You want readable, maintainable object creation,** especially when dealing with domain models or configuration objects. The fluent interface style improves clarity and flexibility.

When to Avoid?

The Builder Pattern can be overkill in simpler use cases. Avoid it when:

- **Your class has only 1-2 fields:** Using a constructor or setter methods is simpler and more concise.
- **You don't need object customization or immutability:** If the object is small, mutable, or built only in one place, a builder adds unnecessary complexity.

Pros and Cons of Builder Pattern

Understanding both the advantages and limitations of the Builder Pattern helps in deciding when to use it effectively.

Pros

- **Avoids constructor telescoping**: You no longer need to write multiple overloaded constructors for different configurations.
- **Ensures immutability**: The final object can be made immutable once built, which improves safety and thread-safety.
- **Clean, readable object creation**: The fluent API makes object construction expressive and easy to follow.
- **Great for complex configurations**: If your object has many optional parameters or conditional setup, the builder pattern keeps it organized.

Cons

- **Slightly tough to set up:** Initial setup requires writing a separate builder class, which adds to boilerplate.
- **Overkill for small classes:** If a class only has one or two fields, using a builder adds unnecessary complexity.
- **Separate builder class needed:** You need to maintain a second class or static inner class just to construct the main object, increasing maintenance.

Real World Products Using Builder Pattern

Understanding where the Builder Pattern is used in real products helps solidify its relevance. Let's look at two real-world examples:

1. Lombok's `@Builder` Annotation (Java)

Lombok is a Java library that reduces boilerplate code using annotations. One of its popular features is the @Builder annotation, which automatically generates a builder class behind the scenes.

Instead of writing the builder logic manually, you just annotate your class:

```
@Builder
public class User {
    private String name;
    private int age;
    private String address;
}
```

Now, you can build objects using a fluent API:

```
User user = User.builder()
        .name("John")
        .age(30)
        .address("NYC")
        .build();
```

---

2. Amazon Cart Configuration

Think about **Amazon's shopping cart system**. When you add an item to your cart, you're not just storing an item ID. You're building a complex object with fields like:

- Quantity
- Size or color (for apparel)
- Delivery option
- Gift wrap
- Save for later status
- Discounted price or offer tag

Each user may customize these options differently. Internally, such cart items are likely created using a **Builder Pattern** to allow step-by-step configuration while ensuring data consistency and immutability.